Pretende-se desenvolver, recorrendo à biblioteca Winsock, uma aplicação Win32, em modo consola, que execute a seguinte sequência de ações:

1. Cria um socket TCP, designado s;

2. Associa s ao porto de escuta 5005 e coloca-o em modo de escuta;

3. Entra num ciclo em que vai aceitando pedidos de ligação TCP em s e, em simultâneo, aguardando por caracteres em ligações TCP já estabelecidas. Recorre, para o efeito, à função select();

4. Quando uma nova ligação TCP é estabelecida, acrescenta o *socket* correspondente à tabela socksTab e envia, através de todos os *sockets* existentes nessa tabela, a *string* «+OK x clients connected\r\n», sendo x o número total de ligações estabelecidas;

5. Quando recebe dados numa das ligações TCP estabelecidas, reencaminha-os pelos restantes *sockets* e envia à origem uma *string* com conteúdo igual a «+OK sent to x clients\r\n», sendo x o número de *sockets* pelos quais os dados foram reencaminhados com sucesso;

6. Quando ocorre um erro no acesso a um dos *sockets* existentes, retira-o da tabela;

7. Quando uma das ligações TCP estabelecidas é encerrada, retira o *socket* correspondente da tabela;

8. Quando ocorre um erro na função select() ou no *socket* s, encerra todos os *sockets* e termina a aplicação;

9. Quando ocorre um *timeout* de TIMEOUT segundos na função *select()*, envia, através de todas as ligações TCP estabelecidas, a *string* "+OK x client(s) connected\r\n", sendo x o número de ligações estabelecidas.

Complete as partes em falta no código fonte. Deve identificar as alíneas ([a], [b], …, [j]) correspondentes às respostas que apresentar.

```
#define LISTENING_PORT 5005
#define TIMEOUT  10 //sec
#define TAB_SIZE 10
#define MAX_SIZE 1000

SOCKET socksTab[TAB_SIZE];
int count = 0;

void Abort(char* msg, SOCKET s);
void sendMsgToClients(char* msg, SOCKET exceptSock);
int addSockToTab(SOCKET s);
int removeSockFromTab(SOCKET s);
void closeSocks();

int main(int argc, char* argv[]) {
      SOCKET s = INVALID_SOCKET, cli;
      int iResult, nbytes;
      struct sockaddr_in addr;
      char message[MAX_SIZE];
      WSADATA wsaData;

      struct timeval timeout;

      fd_set set;

      iResult = WSAStartup(MAKEWORD(2, 2), &wsaData);

      if (iResult != 0) {
            printf("WSAStartup failed: %d\n", iResult);
            exit(EXIT_FAILURE);
      }
/* [a] Cria o socket TCP s */
      if (/*. . .*/)
            Abort("Unable to create socket s", s);

      memset((char*)&addr, 0, sizeof(addr));
      addr.sin_family = AF_INET;


/* [b] Associa o socket s ao porto LISTENING_PORT e configure-o para escuta */
      /*. . .*/

      count = 0;
      timeout.tv_sec = TIMEOUT; //10 seconds
      timeout.tv_usec = 0;

      while (1) {


/* [c] Configura o conjunto set para que inclua o socket s e os count sockets
existentes no array socksTab */
            /*. . .*/


/* [d] Recorre à função select() para aguardar que, pelo menos, um dos sockets
definidos no conjunto set esteja pronto para leitura (ou seja, em que uma
operação de leitura não irá bloquear), até um «tempo de espera» definido pela
variável timeout */

            switch (select(32, &set, NULL, NULL, &timeout)) {
            case /*. . .*/: // error
                  Abort("Error on select", s);
                  break;
            case /*. . .*/: //timeout
```

```
                sprintf_s(message, MAX_SIZE, "+OK %d client(s) connected\r\n", count);
                sendMsgToClients(message, INVALID_SOCKET);
                break;

          case /*. . .*/:
```

/* [e] Se existir um pedido de ligação em s, processa-o de forma adequada */

```
                if /*. . .*/ {
                      Cli = /*. . .*/;

                      if (/*. . .*/)
                            Abort("Error accepting a new connection", s);

                      if (!addSockToTab(cli)) {
                            closesocket(cli);
                      } else {
                            sprintf_s(message, MAX_SIZE, "+OK %d client(s) connected\r\n",
count);
                            sendMsgToClients(message, INVALID_SOCKET);
                      }
                }
```

/* [f] Para cada um dos count sockets existentes, em socksTab, verifica se está pronto para leitura e processa-o de uma forma adequada */

```
                for (int i = 0; i < count; i++) {
                      if (/*. . .*/) {
                            nbytes = /*. . .*/
```

/* [g] Se a operação de leitura resultar num erro, ou numa indicação de encerramento da ligação TCP, retira o socket da tabela */

```
                                  if (/*. . .*/) {
                                        removeSockFromTab(socksTab[i]);
                                  } else {
                                        message[nbytes] = '\0';
                                        sendMsgToClients(message, socksTab[i]);

                                        sprintf_s(message, MAX_SIZE, "+OK sent to %d client(s)\r\n",
                                                                                    count-1);
```

/* [h] Envia a string «message» através do socket socksTab[i]*/

```
                                        if (/*. . .*/) //error
                                              removeSockFromTab(socksTab[i]);
                                  }
                            }
                } //for
                break;
          }
     } //while


     closesocket(s);
     closeSocks();

     WSACleanup();
     return EXIT_SUCCESS;
}
```

```
//*************************************************************************

void Abort(char* msg, SOCKET s) {
	fprintf(stderr, "Error: <%s> (%d)\n", msg, WSAGetLastError());

	if (s != INVALID_SOCKET)
		closesocket(s);
	closeSocks();
	WSACleanup();
	exit(EXIT_FAILURE);
}

//*************************************************************************

void sendMsgToClients(char* msg, SOCKET exceptSock) {
	for (int i = 0; i < count; i++) {
		if (socksTab[i] != exceptSock) {
/* [i] Envie a string msg através do socket socksTab[i]*/
			if (/*. . .*/) { //error
				if (removeSockFromTab(socksTab[i]))
					i--;
			}
		}
	}
}

//*************************************************************************

int addSockToTab(SOCKET s) {
	if (count >= TAB_SIZE)
		return 0; //false

	socksTab[count++] = s;
	return 1; //true
}


//*************************************************************************

int removeSockFromTab(SOCKET s) {
	for (int i = 0; i < TAB_SIZE; i++) {
		if (socksTab[i] == s) {
			closesocket(s);
			for (int j = i; j < TAB_SIZE - 1; j++) {
				socksTab[j] = socksTab[j + 1];
			}
			count--;
			return 1; //true
		}
	}
	return 0; //false
}

//*************************************************************************

/* [j] Encerre os count sockets existentes em socksTab */
void closeSocks() {
	/*. . .*/

	count = 0;
}
```